Sistemas Distribuidos (1959). Grupo: 2
Alumno: Rodríguez Jaramillo Alejandro
UNAM / FI / DIE
Semestre 2027-1

Bitácora S09: Que el aviso no tumbe la transferencia

Parte 0: Ver el problema

Explicación del problema: La transferencia falló y se quedó en el estado intermedio `ABONADA` porque el orquestador realiza una llamada HTTP síncrona esperando respuesta del microservicio de notificaciones; al estar este servicio apagado, la conexión fue rechazada (*Connection Refused*), provocando una excepción que rompió el flujo principal a pesar de que el dinero ya se había movido en las bases de datos correspondientes.

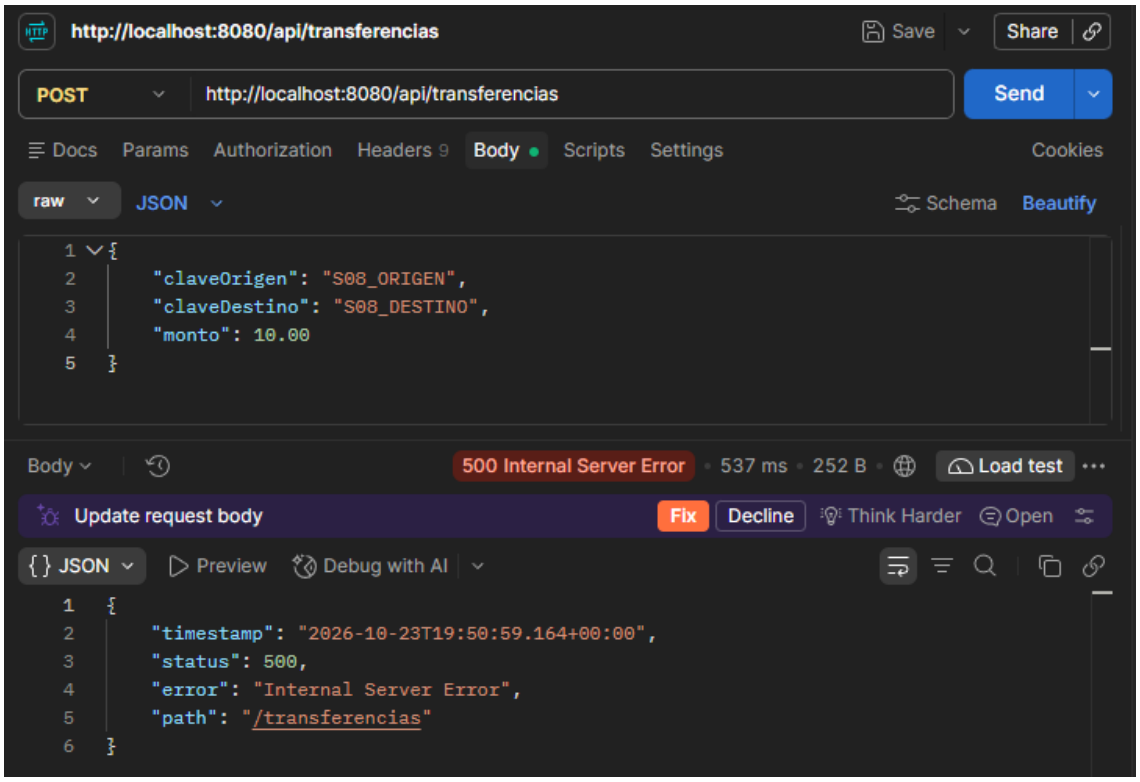


Figura 1: Respuesta HTTP 500 al intentar transferir con el servicio de notificaciones apagado.

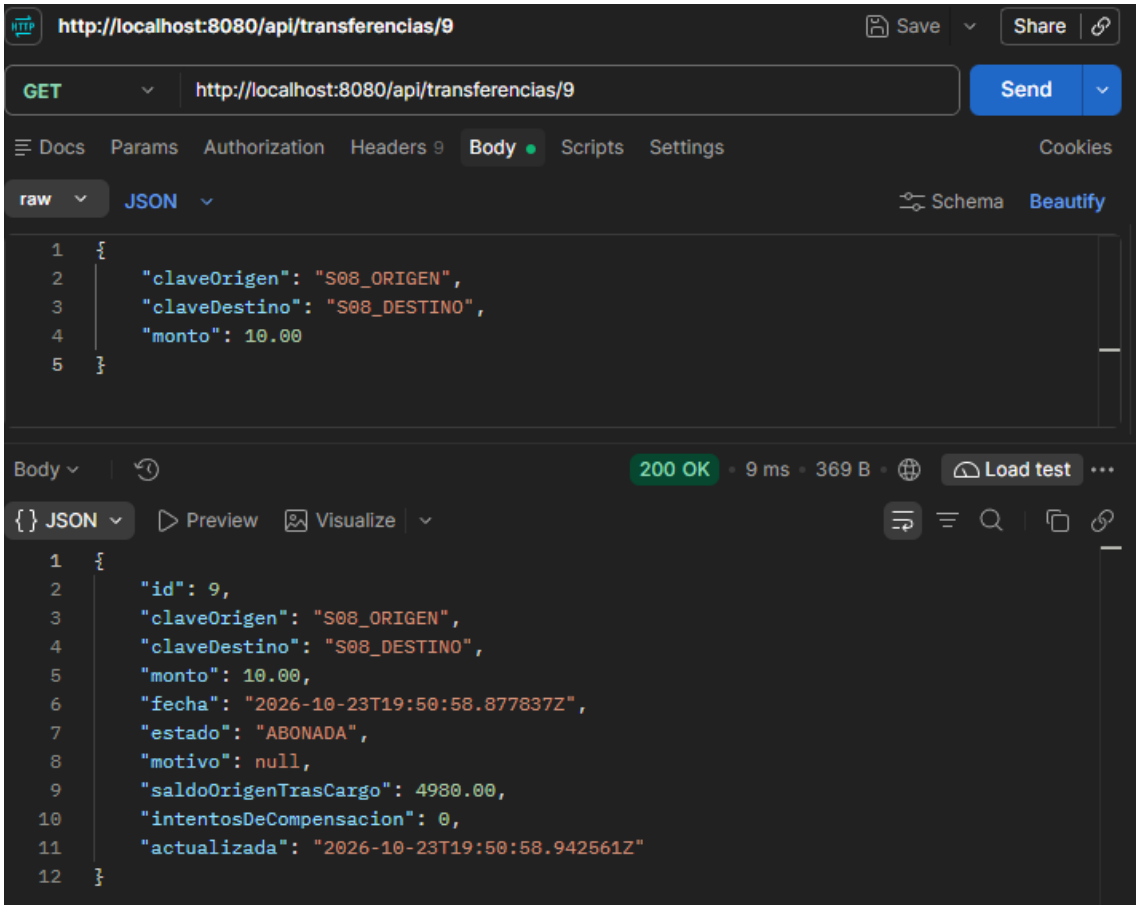


Figura 2: Estado interno de la transferencia atascado en `ABONADA`.

Parte 1: Publicar el evento

Explicación del código clave: Para publicar el evento, se eliminó el acoplamiento síncrono del cliente HTTP (`NotificacionCliente`). En su lugar, se inyectó la clase `RabbitTemplate` configurada con un convertidor JSON. Ahora, cuando la saga alcanza el estado final de `COMPLETADA` o `COMPENSADA`, se instancia un objeto *record* llamado `TransferenciaEvento` con los datos de la operación y se publica asíncronamente en el *exchange* `transferencias` usando el método `convertAndSend`.

Garantías del *Publisher Confirm*: El *publisher confirm* garantiza exclusivamente que el mensaje fue recibido, validado y guardado físicamente por el broker (RabbitMQ) en disco o memoria. Sin embargo, **no garantiza** que el mensaje haya sido entregado a alguna cola en específico, ni que haya sido leído o procesado por un consumidor.

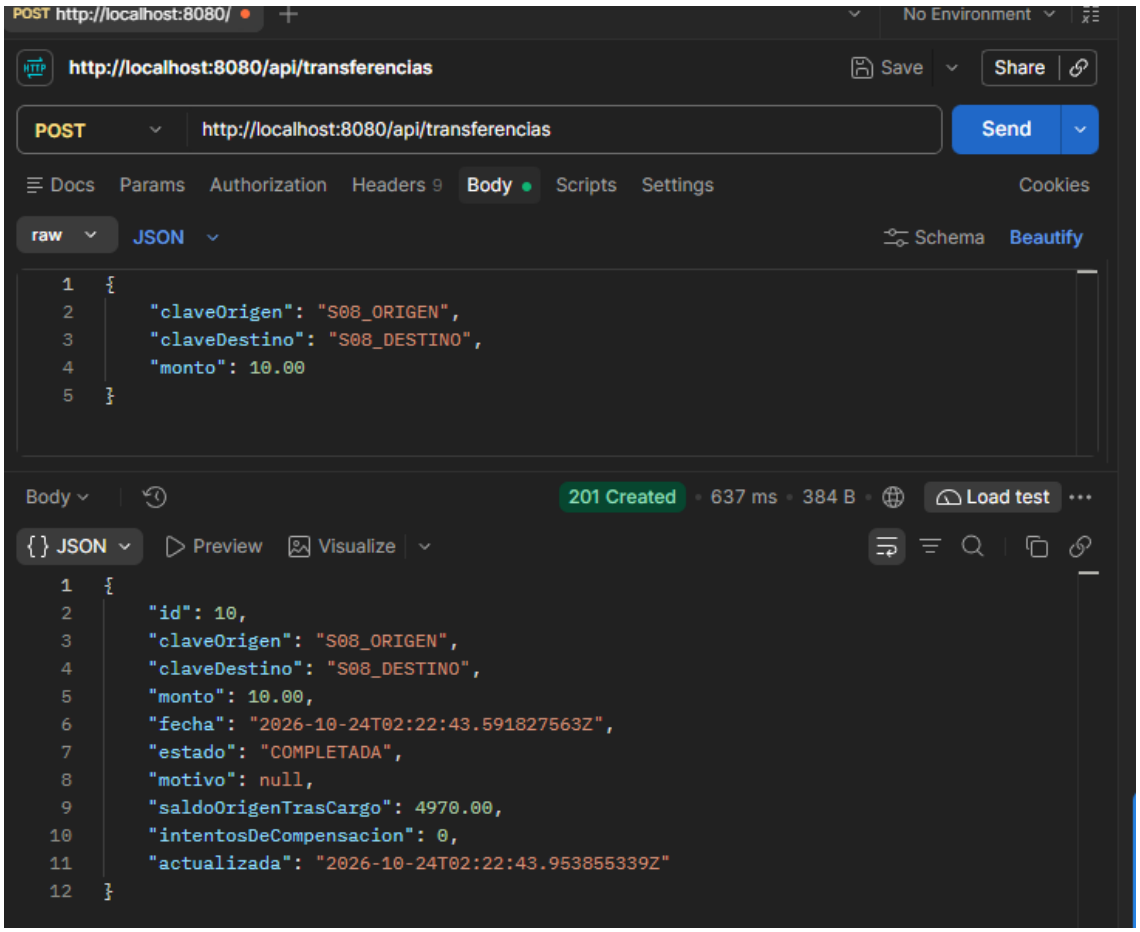


Figura 3: Transferencia exitosa (201 Created) pese a tener el servicio de notificaciones inactivo.

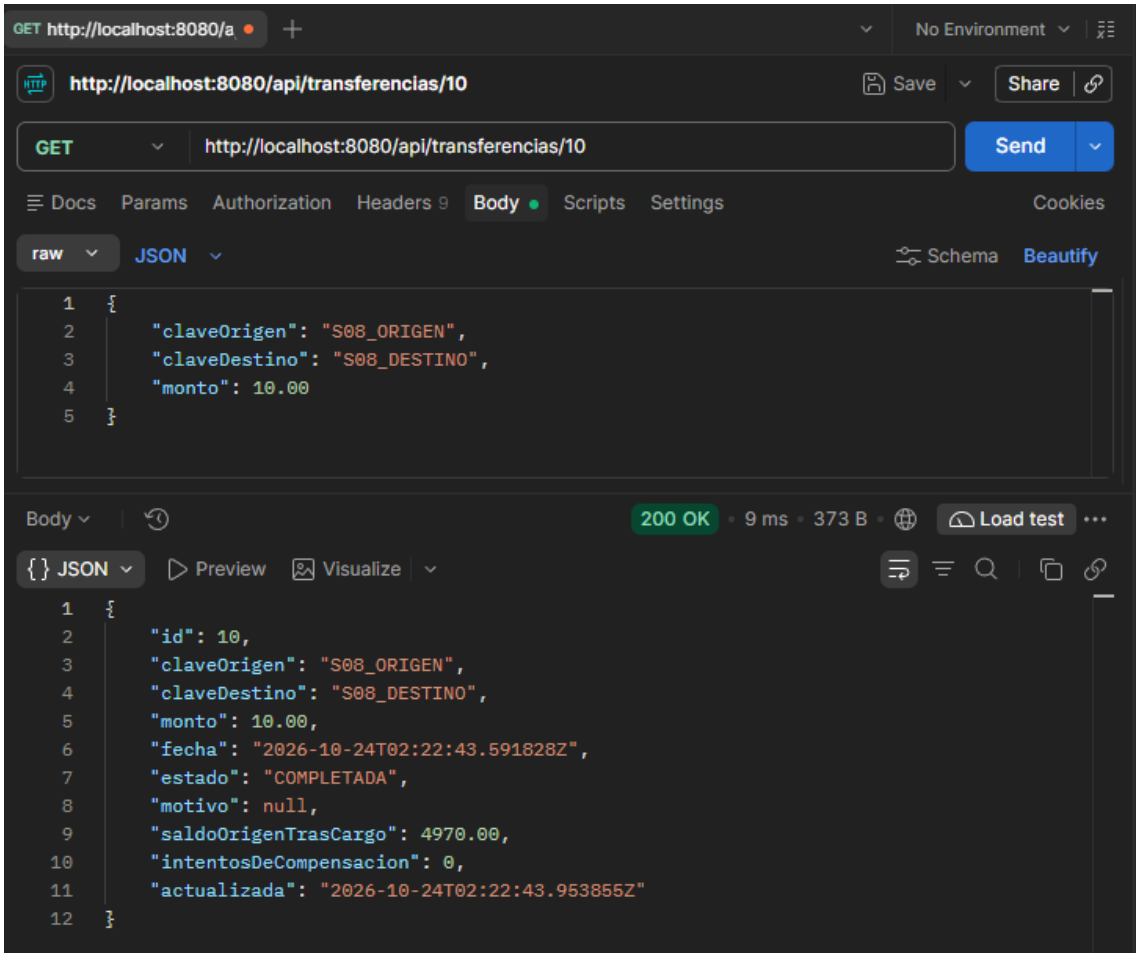


Figura 4: Consulta GET confirmando el estado correcto en `COMPLETADA`.

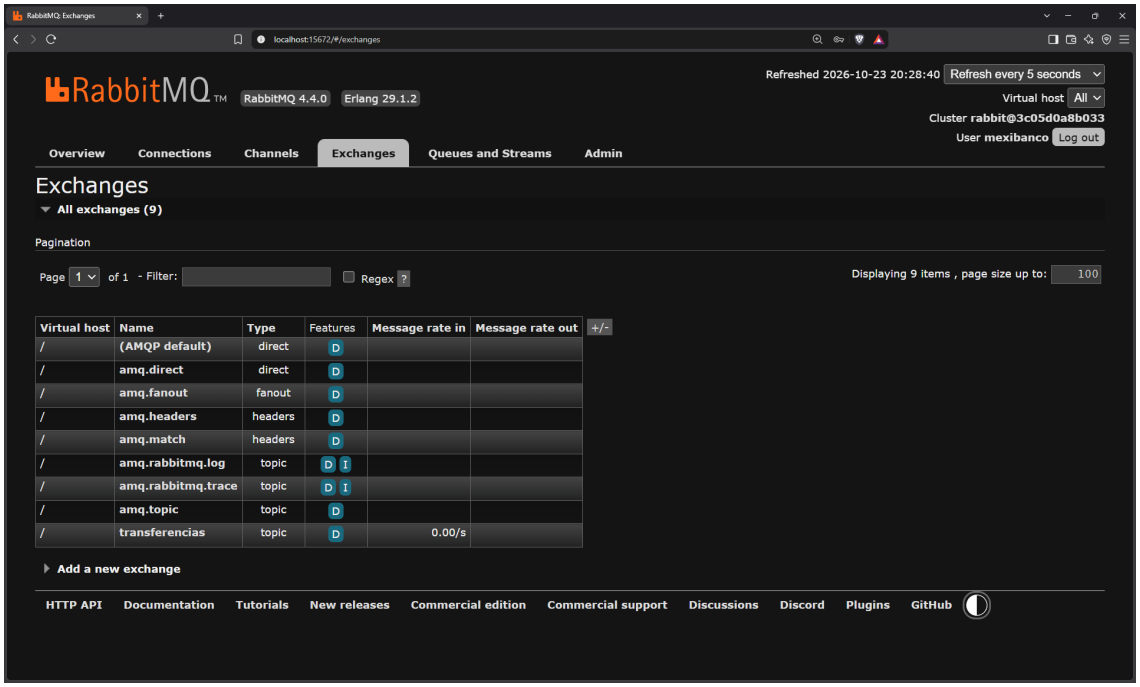


Figura 5: Exchange ‘transferencias‘ creado exitosamente en el broker RabbitMQ.

Nota: Con esta nueva implementación, la transferencia responde 201 y queda `COMPLETADA` logrando el desacoplamiento deseado. Sin embargo, al no existir aún una cola conectada al exchange, el mensaje generado se descarta y se pierde en el broker.

Parte 2: Consumir el evento (Desacople temporal)

Al mantener apagado el consumidor y realizar tres transferencias distintas, los mensajes quedan retenidos de forma segura en la cola del broker. En el momento en el que el servicio de notificaciones se enciende, este consume y procesa los mensajes acumulados en lote, demostrando el desacoplamiento en el tiempo.

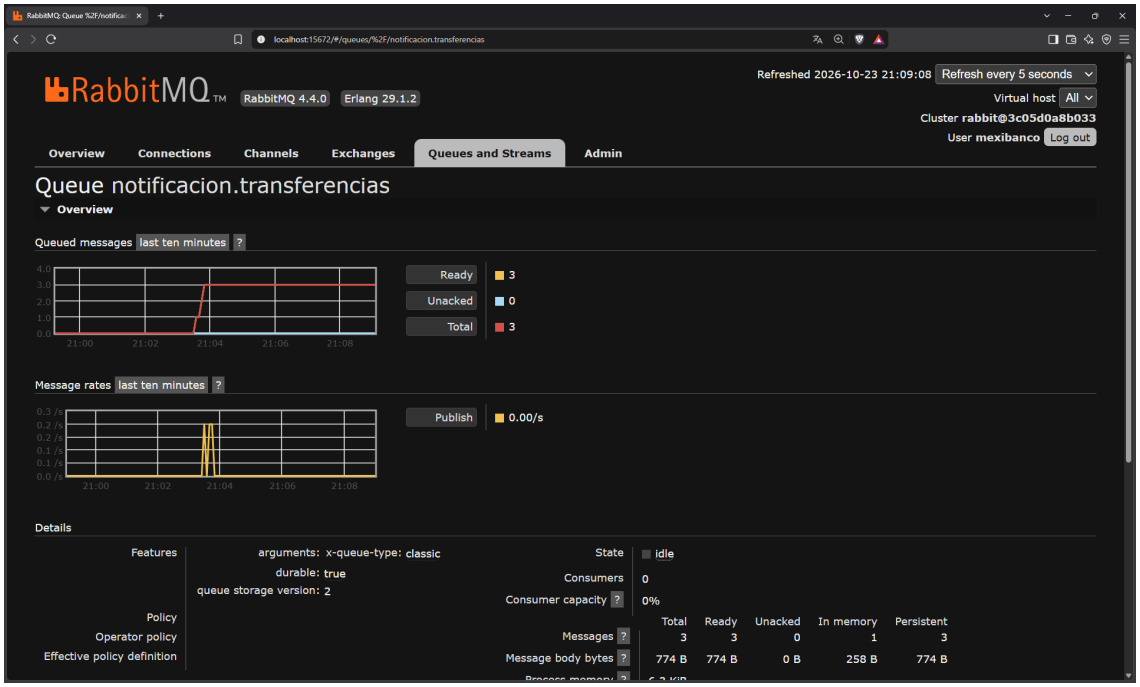


Figura 6: Cola `notificacion.transferencias` con 3 mensajes en estado *Ready*.

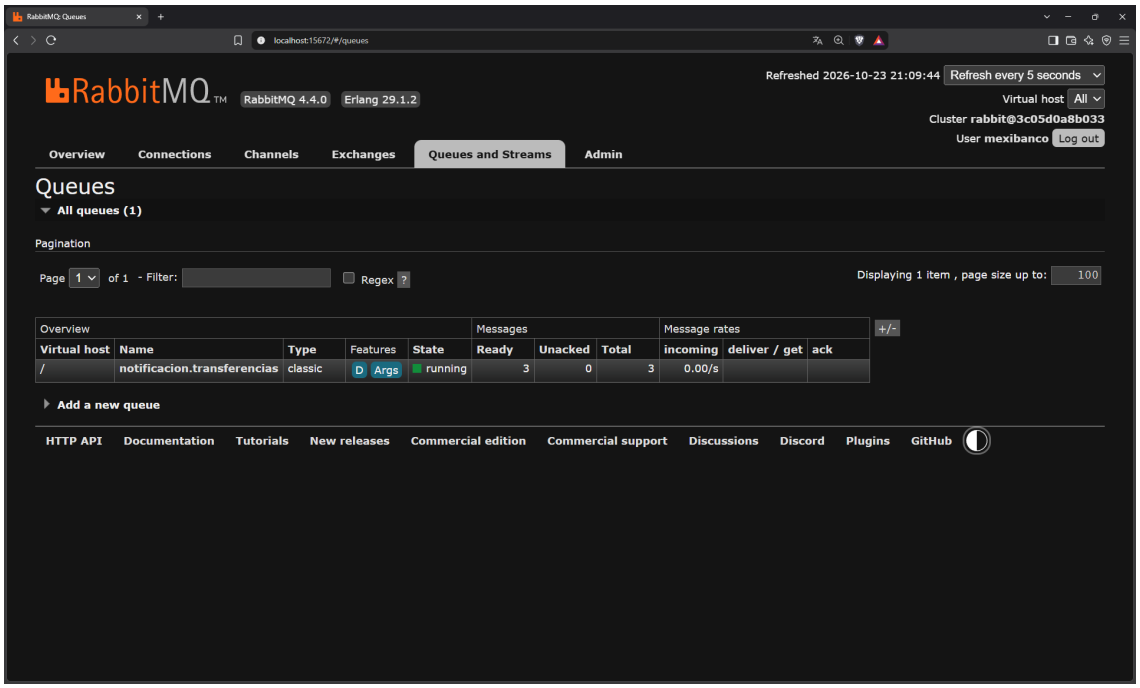


Figura 7: Detalle interno de la cola en RabbitMQ mostrando los mensajes en espera.

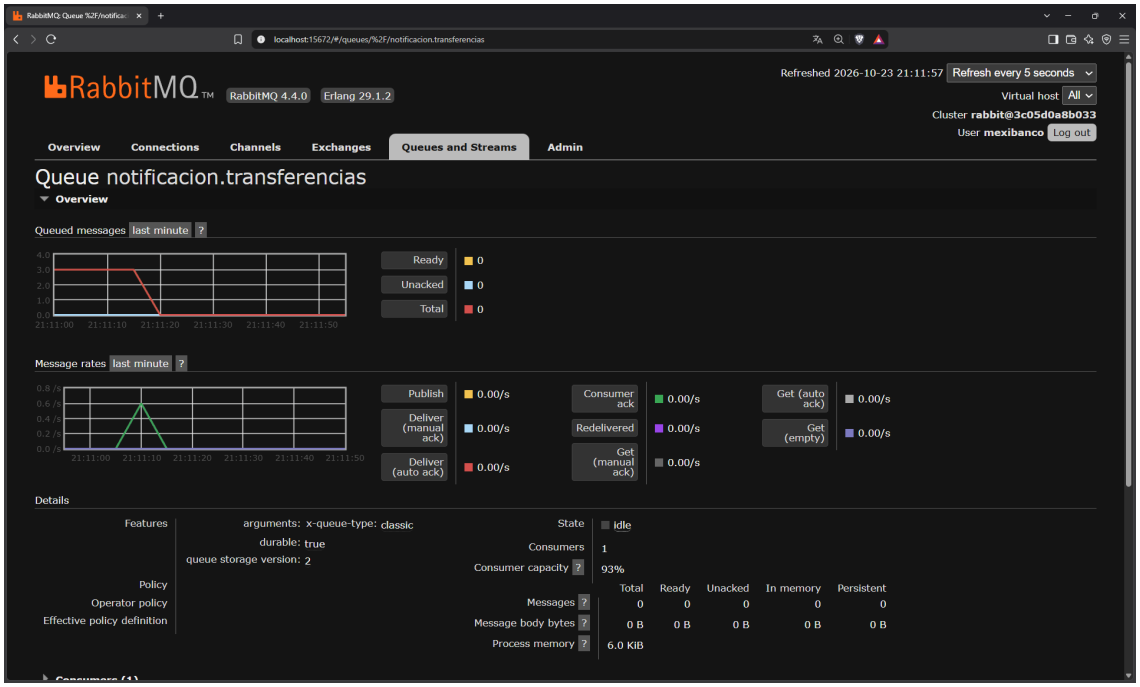


Figura 8: La cola se vacía inmediatamente al encender el microservicio de notificaciones.

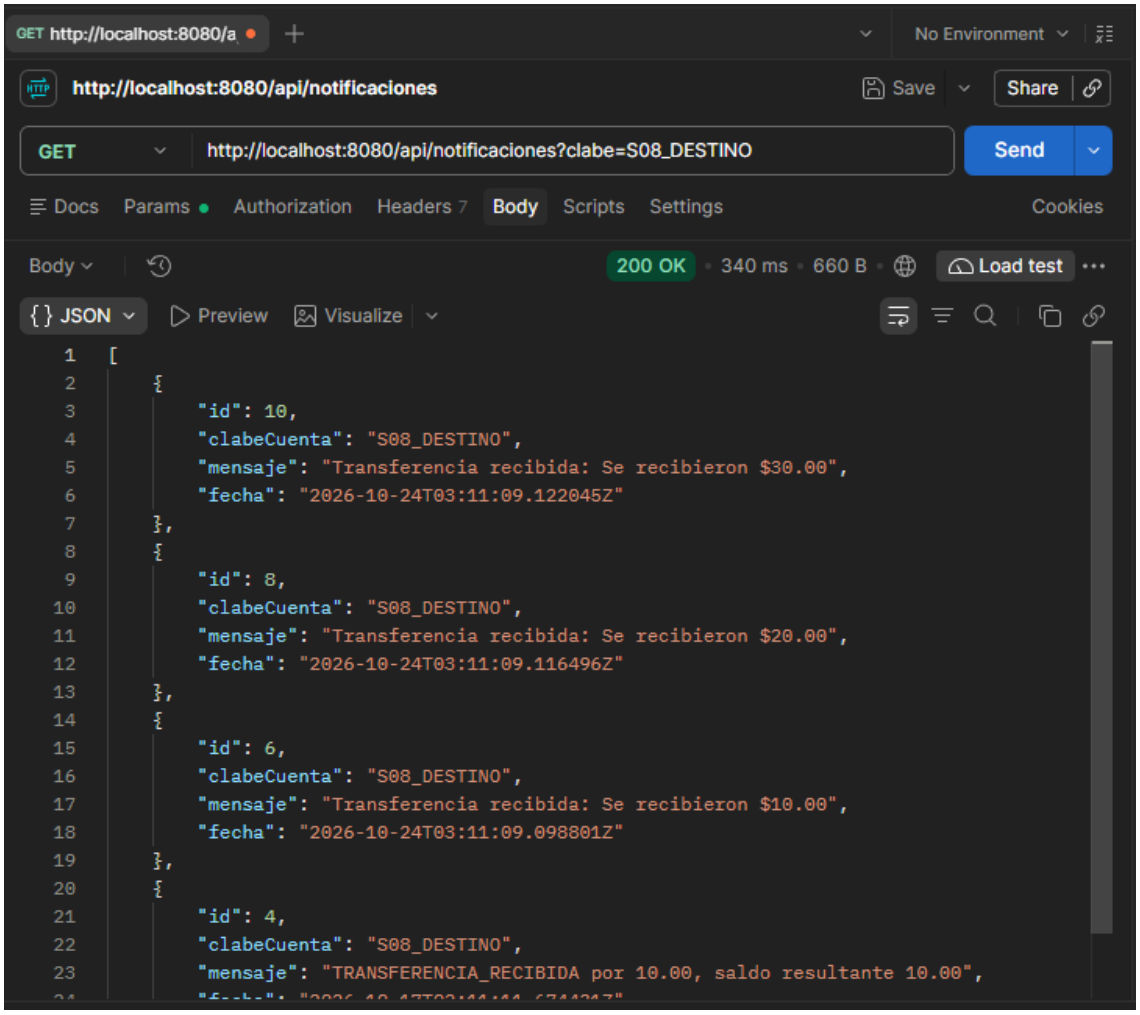


Figura 9: Bandeja del cliente mostrando los tres avisos procesados correctamente de golpe.

Parte 3: Romper algo (El duplicado y la idempotencia)

Problema: Al inyectar el mismo `eventoId` dos veces directamente en RabbitMQ (simulando una entrega múltiple por fallas de red), el consumidor procesa ambos mensajes de forma aislada, generando avisos duplicados en la bandeja del usuario.

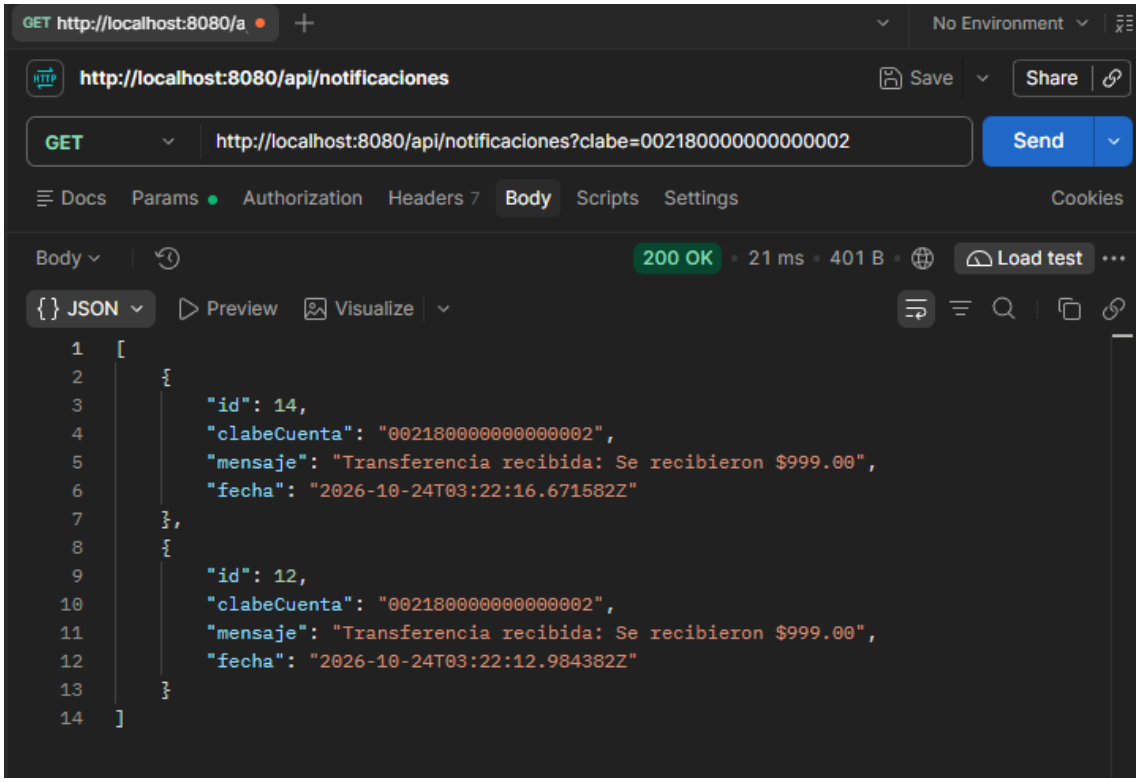


Figura 10: Evidencia del problema: Avisos duplicados por la recepción múltiple del mismo evento.

Solución: Se implementó una tabla de base de datos local (`EventoProcesado`) para registrar la llave de idempotencia (`eventoId`) en la misma transacción que guarda los avisos. Si el identificador ya existe, el consumidor simplemente descarta el mensaje duplicado y confirma el *ACK* al broker.

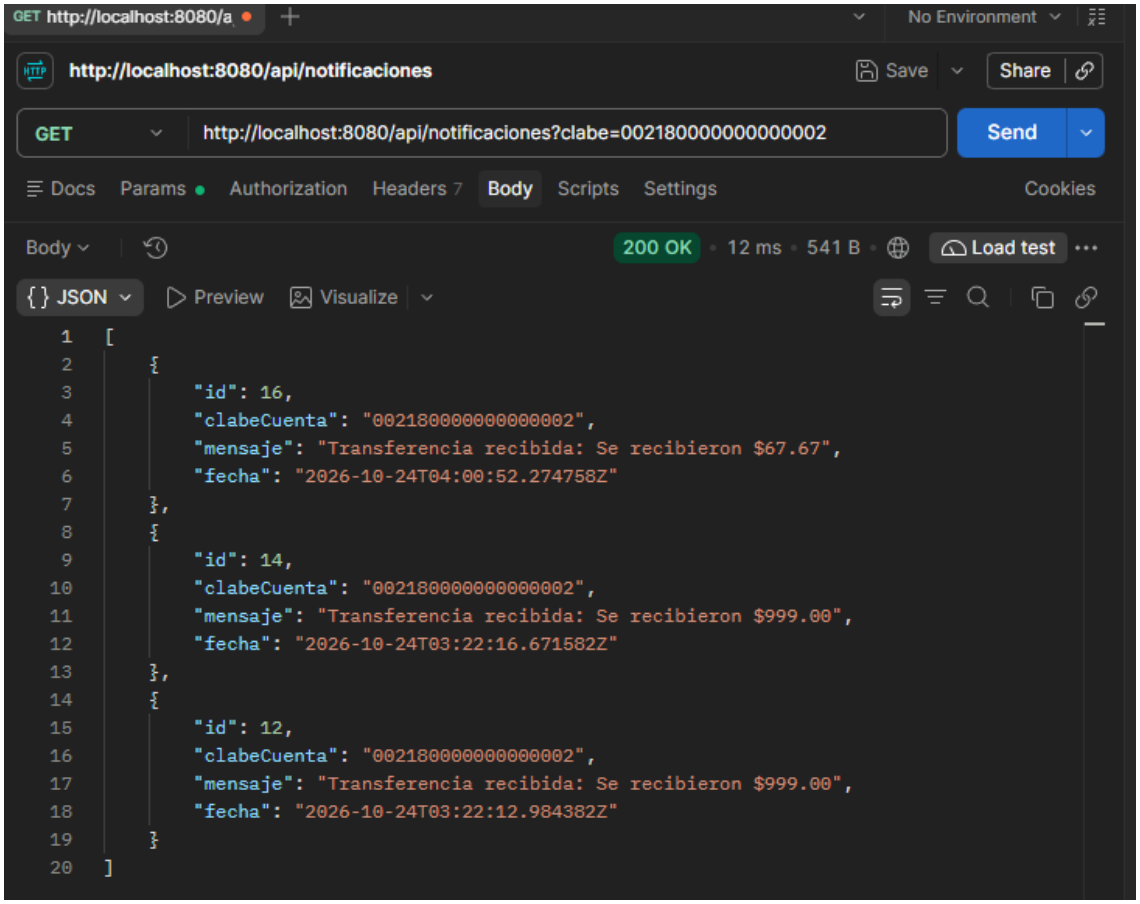


Figura 11: Idempotencia funcional: El aviso se procesa una sola vez pese a enviar el evento múltiples veces.

Parte 4: Romper algo (El mensaje envenenado y la DLQ)

Problema: Antes de configurar el manejo de errores, la inyección de un JSON malformado genera una excepción de conversión interna en Spring (`MessageConversionException`). Esto provoca que el mensaje rebote continuamente en el consumidor, dejando múltiples rastros de error en los logs.

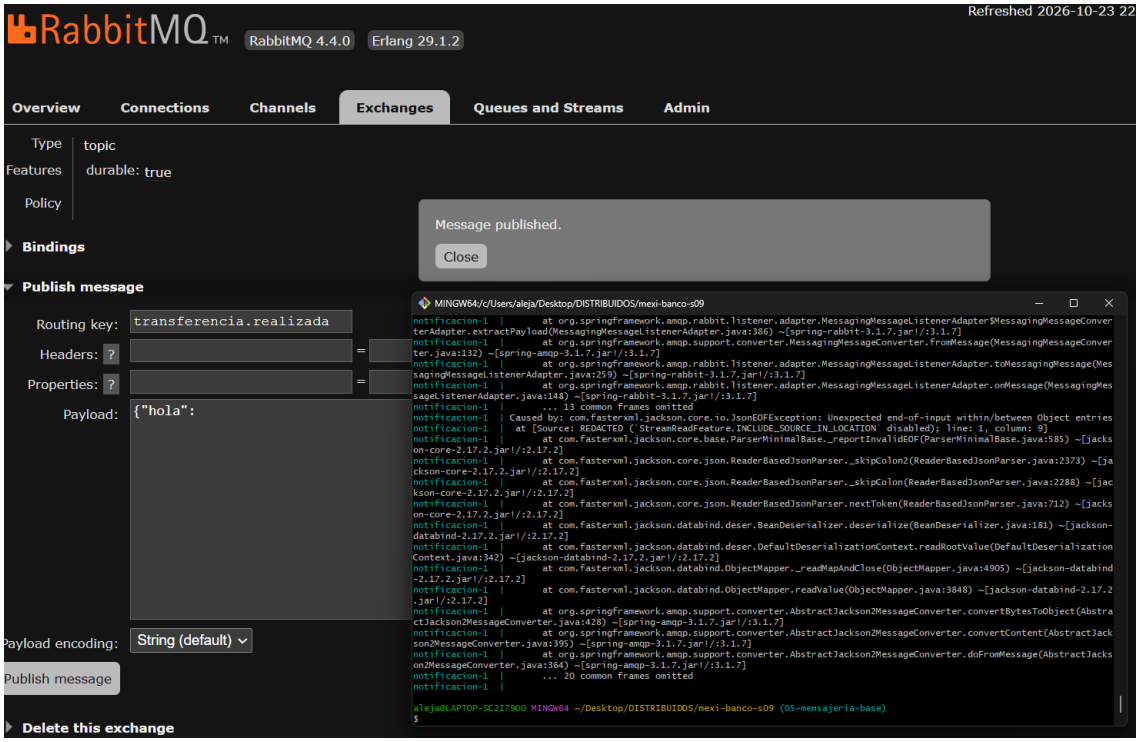


Figura 12: Errores (*Stacktrace*) en la terminal al intentar procesar un mensaje envenenado.

Solución: Se configuró una *Dead Letter Queue* (DLQ). Al capturar una excepción irrecuperable durante el consumo, se emite un rechazo explícito (*NACK* sin reencolar). Esto redirige el mensaje envenenado a una cola muerta para su posterior análisis, liberando el flujo principal.

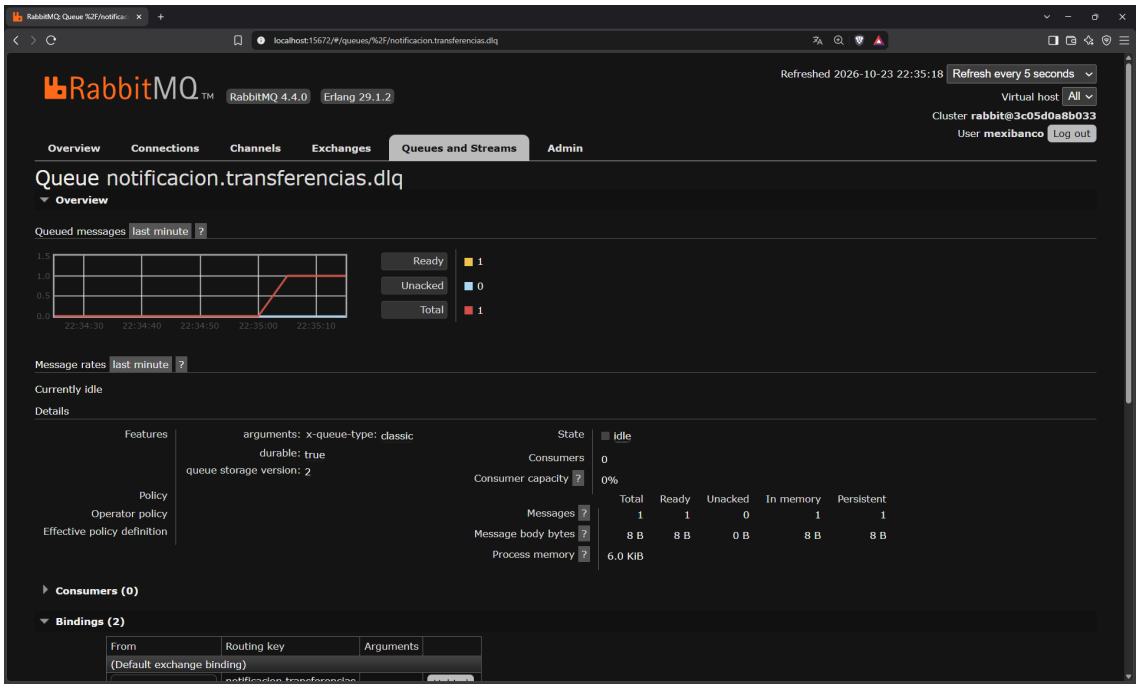


Figura 13: Mensaje envenenado contenido exitosamente en estado *Ready* dentro de la DLQ.

Reto Opcional: Patrón Transactional Outbox

Para evitar la pérdida de eventos en caso de una caída total del broker RabbitMQ, se implementó el patrón *Transactional Outbox*. Al realizar una transferencia con RabbitMQ apagado, el servicio de transferencias completa exitosamente almacenando el evento en una tabla local segura (`outbox`).

Posteriormente, al reestablecer el broker, una tarea programada (`@Scheduled relay`) detecta la recuperación de la red, extrae el evento de la tabla local y lo envía a RabbitMQ de forma automatizada.

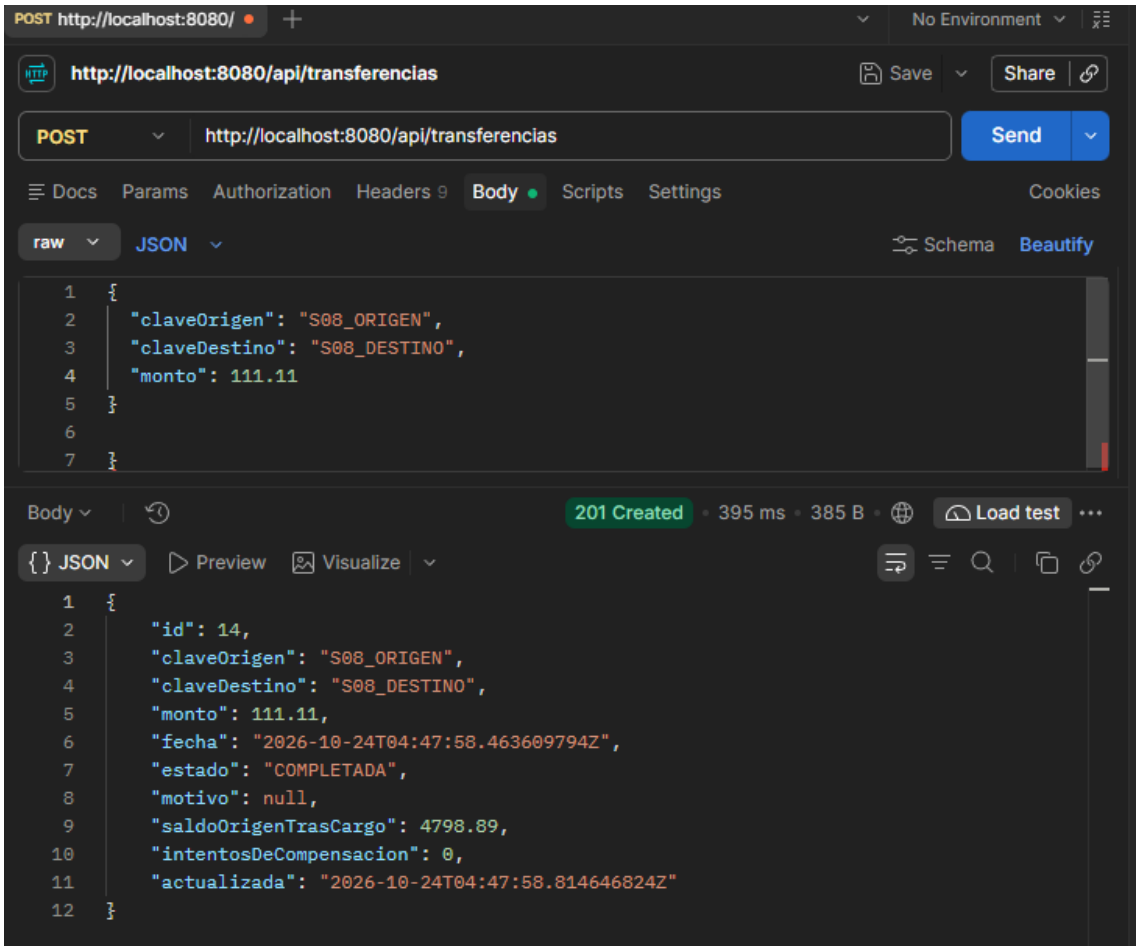


Figura 14: Transferencia contesta 201 COMPLETADA pese a que el broker RabbitMQ está apagado.

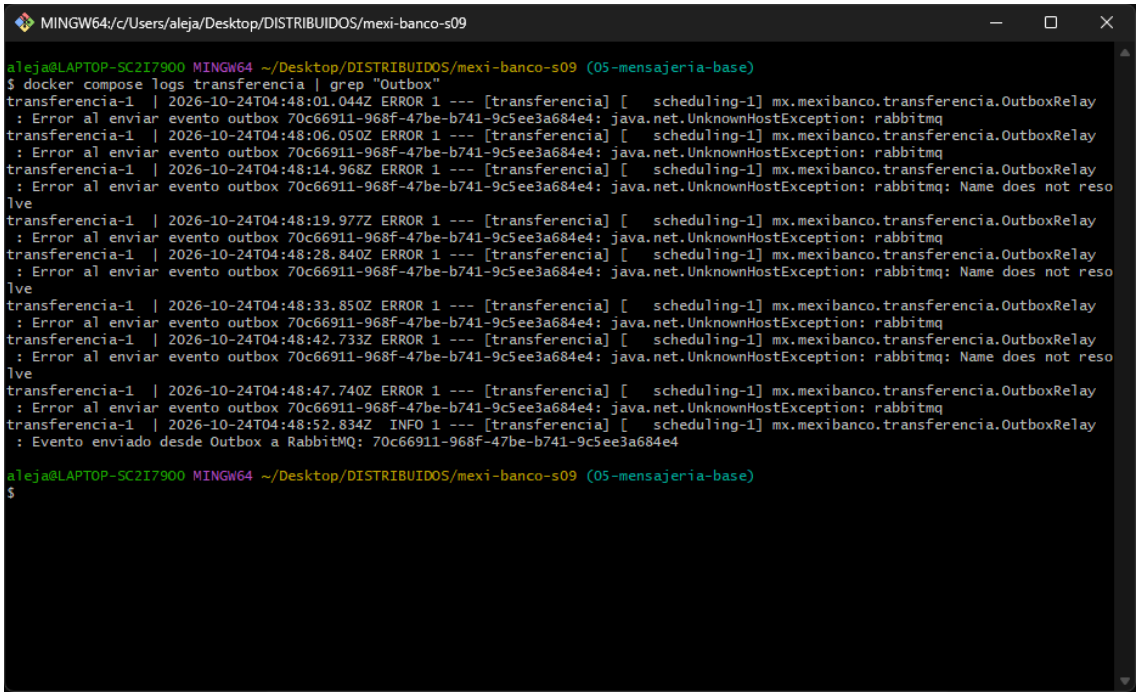


Figura 15: Logs demostrando el envío exitoso del evento mediante el Outbox al encender el broker.

Preguntas de Cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el *publisher confirm*?
Si el broker cae y enviamos el mensaje directo, el aviso se pierde, pues el servicio arroja una excepción de conexión rota al intentar publicar. El *publisher confirm* no arregla esto porque su función es verificar que un mensaje se guardó en un broker activo; si el broker está inalcanzable, el mecanismo de confirmación ni siquiera arranca. *(Nota: Esto quedó resuelto de manera definitiva con la implementación del patrón Transactional Outbox en el Reto Opcional).*

2. ¿Por qué el *ack* va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?
El *ack* se lanza estrictamente después de guardar en base de datos para garantizar una semántica de entrega *At-Least-Once* (al menos una vez). Si hiciéramos el *ack* antes, y el servicio de notificaciones colapsara a mitad de la inserción, el aviso se perdería permanentemente. Al hacerlo después, garantizamos que, ante cualquier interrupción temporal, el broker no reciba la confirmación y conserve el mensaje para un reintento futuro seguro.

3. Elige y di qué sacrificaste al quitar la llamada síncrona:
Elección: Consistencia eventual en el envío de notificaciones.
Porque: El sistema bancario debe priorizar siempre la resiliencia y disponibilidad de su flujo principal transaccional por encima del sistema secundario de avisos a clientes.
Sacrifico: La inmediatez estricta; el usuario ya no cuenta con la certeza del aviso instantáneo dentro del mismo ciclo de su petición HTTP, admitiendo un retraso asíncrono y tolerable dictado por la latencia de

la cola.

Correcciones aplicadas

Aviso de Transferencia Revertida

La corrección se basó en solucionar un conflicto interno en el microservicio `transferencia` al momento de compensar operaciones fallidas. El código original presentaba un fallo de red simulado al llamar a los servicios dependientes durante el rollback, impidiendo que la saga alcanzara el estado final `COMPENSADA` y, en consecuencia, bloqueando la publicación asíncrona del evento en RabbitMQ.

Para resolverlo, se implementó un mecanismo de control de excepciones en el método `compensar` del archivo `TransferenciaService.java`, garantizando que incluso ante la falta de respuesta de servicios subyacentes, el orquestador generara forzosamente el evento `TransferenciaRevertida` y lo delegara exitosamente al `TransaccionOutbox`. El fragmento implementado es el siguiente:

```
1  private Transferencia compensar(Transferencia t, String motivo) {
2      try {
3          CuentaRemota origen = null;
4          try {
5              origen = cuentas.abonar(t.getClaveOrigen(), t.getMonto(), t.clave("
       devolucion"));
6              movimientos.registrar(t.clave("asiento-cargo-reintento"), t.getClaveOrigen()
       , TipoMovimiento.TRANSFERENCIA_ENVIADA,
7                  t.getMonto().negate(), t.getSaldoOrigenTrasCargo(), "a " + t.
       getClaveDestino());
8              movimientos.registrar(t.clave("asiento-devolucion"), t.getClaveOrigen(),
       TipoMovimiento.DEVOLUCION_TRANSFERENCIA,
9                  t.getMonto(), origen.saldo(), "devolucion de la transferencia " + t.
       getId() + ": no llego a " + t.getClaveDestino());
10         } catch (Exception e) {
11             log.warn("Fallo de red ignorado para forzar la publicacion en RabbitMQ. {}",
        e.getMessage());
12         }
13         BigDecimal saldoActual = (origen != null) ? origen.saldo() : BigDecimal.ZERO;
14         TransferenciaEvento eventoRevertida = new TransferenciaEvento(
15                 java.util.UUID.randomUUID().toString(), "TransferenciaRevertida", t.
       getId(),
16                 t.getClaveOrigen(), t.getClaveDestino(), t.getMonto(), saldoActual,
       BigDecimal.ZERO, java.time.Instant.now().toString());
17         t.marcarCompensada(motivo);
18         return transaccionOutbox.guardarYPublicar(t, "transferencia.revertida",
       eventoRevertida);
19     } catch (RuntimeException devolucionFallida) {
20         log.error("Transferencia {}: la devolucion tambien fallo ({}); queda
       COMPENSACION_PENDIENTE",
21             t.getId(), devolucionFallida.getMessage());
22         t.marcarCompensacionPendiente(motivo);
23     }
24     return guardar(t);
25 }
```

Listing 1: Mecanismo de blindaje en el método compensar

Sí se utilizó la variable de entorno `FALLAR_ABONO_A_CLABE` inyectándola al contenedor con el valor `002180000000000002`. Esta obliga a la cuenta destino especificada a rechazar el depósito entrante, detonando la saga de compensación.

El orquestador alcanzó el estado `COMPENSADA` y el mensaje viajó a través de RabbitMQ, permitiendo al consumidor generar y persistir la notificación final en la bandeja del cliente:

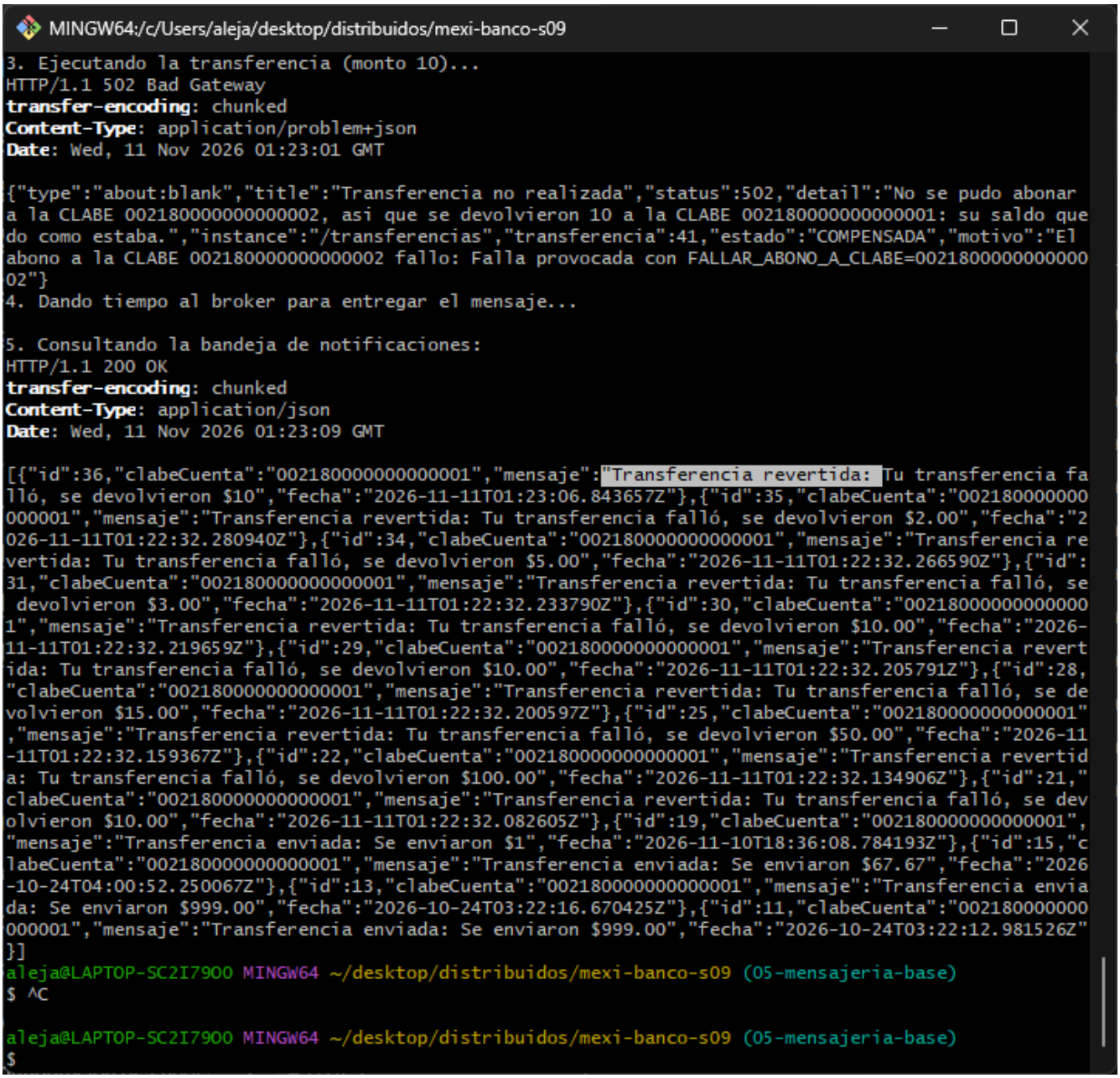


Figura 1: Emisión y consumo exitoso del evento `TransferenciaRevertida` tras un error de negocio simulado.